

μProcessador 2.1 Gravação do Circuito numa FPGA

Os circuitos construídos em VHDL podem ser materializados no mundo real através de algum componente programável, como uma FPGA¹. Serão utilizadas as placas **DE10-Lite** da fabricante **terasic**² para construir um circuito exemplo simples que servirá de base para os experimentos com o circuito físico e gravar o microprocessador da equipe ao final do semestre.

Tecnicidades

A placa, que conta com chaves, leds, displays e diversas interfaces mais complexas, pode ser vista no diagrama de blocos a seguir. Num primeiro momento serão usadas apenas os botões, as chaves, LEDs e os displays de 7 segmentos

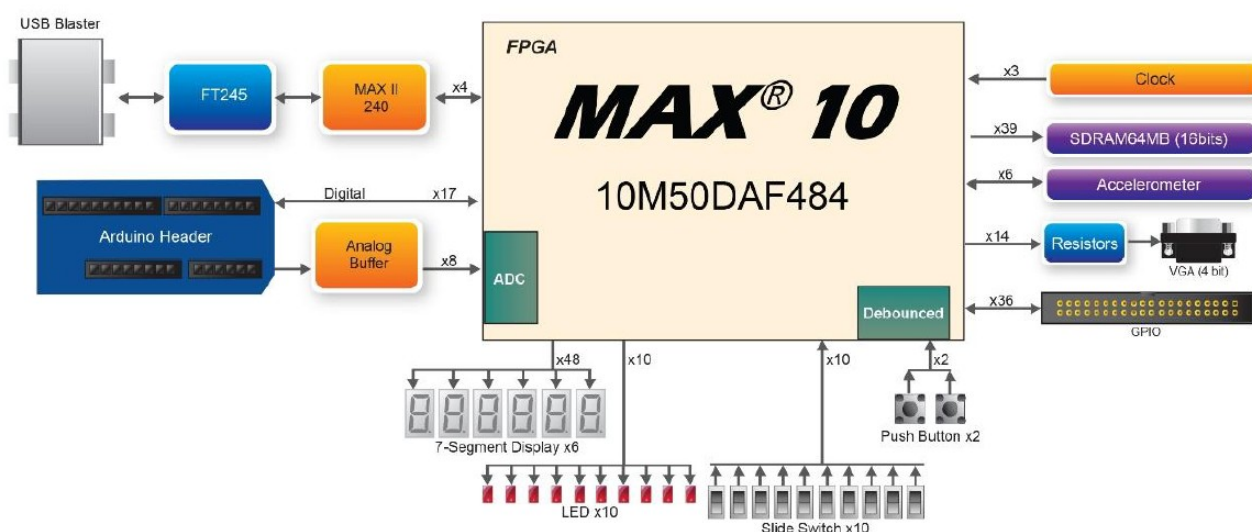


Figure 1-4 Board Block Diagram

Ligue a placa com o cabo USB e se houver algo programado na FPGA o comportamento do circuito já será evidenciado nos LEDs e displays.

Projeto Exemplo

Rode o Quartus Prime instalado em sua máquina (o projeto para a versão 21 pode ser carregado em versões anteriores, a partir da 18). A versão 15 precisa de um projeto específico, porém os arquivos com a descrição do circuito (***.vhd**) e a definição dos pinos (**DE10Lite.qsf**) podem ser importados de uma versão para outra. Abra o projeto exemplo disponibilizado e visualize os dois arquivos fonte VHDL clicando nos seus nomes no *Project Navigator*, aba *Files*.

1 *Field Programming Gate Array*, são os circuitos programáveis mais poderosos disponíveis atualmente.

2 A terasic fabrica as placas de avaliação com FPGAs da Altera, que é parte da Intel agora;

Um dos arquivos, *displays.vhd*, implementa a funcionalidade para converter um número para o formato 7 segmentos dos displays:. Sua interface é:

```
entity displays is
  port (
    dado_in      : in unsigned (7 downto 0); -- numero binario de entrada
    disp0_out    : out unsigned(6 downto 0); -- display convertido para 7 seg
    disp1_out    : out unsigned(6 downto 0); -- display convertido para 7 seg
    disp2_out    : out unsigned(6 downto 0); -- nao usado
    disp3_out    : out unsigned(6 downto 0); -- nao usado
    disp4_out    : out unsigned(6 downto 0); -- nao usado
    disp5_out    : out unsigned(6 downto 0); -- nao usado
  );
end entity;
```

Um número escrito nos pinos *dado_in* será convertido para as saídas que são mapeadas no arquivo *toplevel.vhd* para acendimento em dois displays de 7 segmentos; A conversão tem dois passos em VHDL, que incluem uso de código não coberto nos pdfs da disciplina, mas apenas consistem em converter binário para BCD e então BCD para 7 segmentos.

O outro arquivo, *toplevel.vhd*, faz as ligações do decodificador do display para as chaves e demais pinos de hardware da placa (pinos com sufixo *_HW*). Os pinos de *clock* e *reset* serão usados mais adiante nos circuitos síncronos.

```
entity toplevel is
  port (
    --- clock master da placa ligado na FPGA
    CLK_H_HW   : in std_logic;           -- CLOCK PIN_N14 (50 MHz)
    -- sinais que sao a interface de teste no HW fisico
    RST_HW     : in std_logic;           -- KEY0: PIN_B8
    KEY1_HW    : in std_logic;           -- KEY1: PIN_A7

    SWITCH_HW  : in unsigned (9 downto 0); -- SW9..SW0
    LED_HW     : out unsigned (9 downto 0); -- LED9..LED0
    -- displays da placa conectados na FPGA
    HEX0_HW    : out std_logic_vector(6 downto 0); -- display 7 segmentos (LSd)
    HEX1_HW    : out std_logic_vector(6 downto 0); -- display 7 segmentos
    HEX2_HW    : out std_logic_vector(6 downto 0); -- display 7 segmentos
    HEX3_HW    : out std_logic_vector(6 downto 0); -- display 7 segmentos
    HEX4_HW    : out std_logic_vector(6 downto 0); -- display 7 segmentos
    HEX5_HW    : out std_logic_vector(6 downto 0); -- display 7 segmentos (MSd)
  );
end entity;
```

Para gerar o arquivo de gravação é necessário especificar quais pinos lógicos do nosso projeto (os *in* e *out*, sempre da nossa entidade *top-level*) devem ser ligados em quais pinos físicos da FPGA. O nível de arquivo do *Test Bench* é substituído pela interação física com a placa.

Estas ligações são feitas usando-se o *Assignment Editor* (procure no menu *Assignment*), como visto na próxima figura, ou o *Pin Planner*, que mostra todos os pinos. Perceba que isso é apenas uma tabela que diz onde ligar o quê. No mesmo menu é possível importar a uma definição de pinos de um arquivo *.qsf*

Assignment Editor - D:/LAB-FPGA/2022-1/FPGAEx1-rev2 Q21/FPGAEx1-rev2 - test1						
File Edit View Tools Window Help						
<<new>>		Filter on node names: *		Category: All		
	tatu	From	To	Assignment Name	Value	Enabled
50	✓		out LED_HW[5]	Location	PIN_C13	Yes
51	✓		out LED_HW[6]	Location	PIN_E14	Yes
52	✓		out LED_HW[7]	Location	PIN_D14	Yes
53	✓		out LED_HW[8]	Location	PIN_A11	Yes
54	✓		out LED_HW[9]	Location	PIN_B11	Yes
55	✓		in RST_HW	Location	PIN_B8	Yes
56	✓		in SWITCH_HW[0]	Location	PIN_C10	Yes
57	✓		in SWITCH_HW[1]	Location	PIN_C11	Yes
58	✓		in SWITCH_HW[2]	Location	PIN_D12	Yes
59	✓		in SWITCH_HW[3]	Location	PIN_C12	Yes
60	✓		in SWITCH_HW[4]	Location	PIN_A12	Yes
61	✓		in SWITCH_HW[5]	Location	PIN_B12	Yes
62	✓		in SWITCH_HW[6]	Location	PIN_A13	Yes
63	✓		in SWITCH_HW[7]	Location	PIN_A14	Yes
64	✓		in SWITCH_HW[8]	Location	PIN_B14	Yes
65	✓		in SWITCH_HW[9]	Location	PIN_F15	Yes
66		<<new>>	<<new>>	<<new>>		

Se observarmos o manual da placa (*DE10-Lite_User_Manual.pdf*) podemos ver o nome dos pinos da FPGA e onde eles estão ligados.

Por exemplo, se quisermos ligar o bit RST_HW do nosso projeto ao *pushbutton* KEY[0] da placa, olhamos na pág. 25, tabela 3.3 (ver a seguir) e descobrimos que o botão que queremos está ligado ao pino B8 da FPGA. Portanto foi inserida uma linha para fazer essa associação.

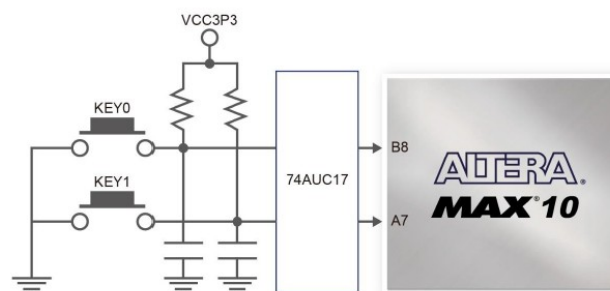


Figure 3-13 Connections between the push-button and MAX 10 FPGA

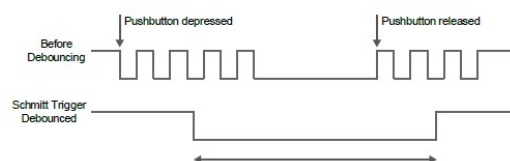


Figure 3-14 Switch debouncing

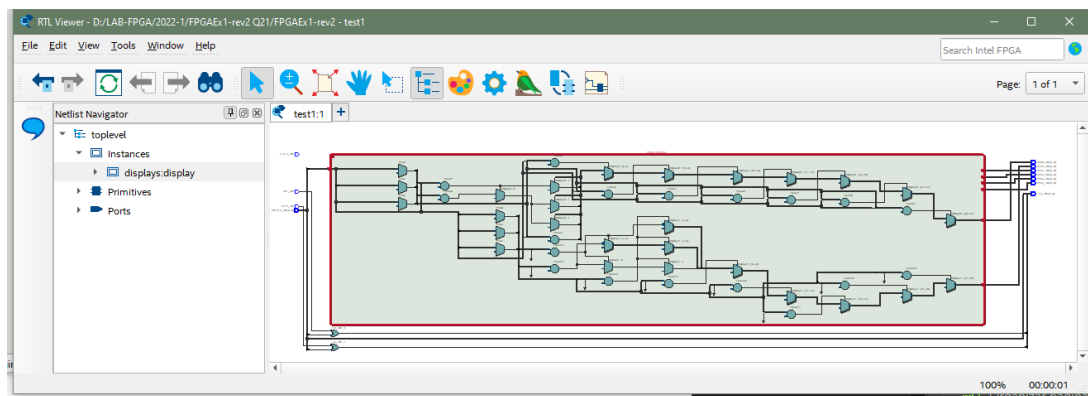
Table 3-3 Pin Assignment of Push-buttons

Signal Name	FPGA Pin No.	Description	I/O Standard
KEY0	PIN_B8	Push-button[0]	3.3 V SCHMITT TRIGGER*
KEY1	PIN_A7	Push-button[1]	3.3 V SCHMITT TRIGGER*

Descarregando o Projeto na Placa

Primeiramente devemos compilar o código VHDL e gerar o mapeamento para a FPGA. Para isso, pressione o botão do “Play” ou vá em *Processing* => *Start Compilation*. Cerca de 40 *warnings* serão gerados devidos principalmente ao nosso mapeamento estar incompleto, mas não deve haver nenhum *error* propriamente dito. Deve levar entre 15 s a 1 min pra terminar.

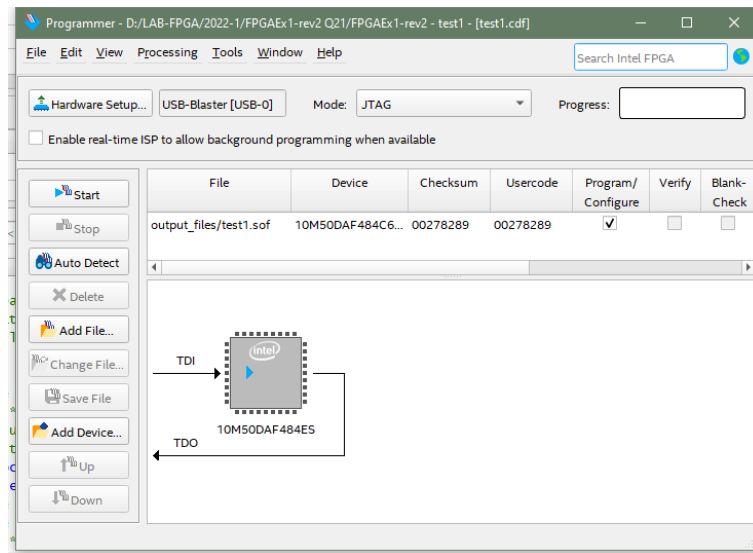
É possível ver o esquemático do circuito gerado usando *Tools* => *Netlist Viewers* => *RTL Viewer*. Isso pode ser útil para o debug, apesar de que as otimizações levam o circuito a ficar um pouco diferente do que normalmente se espera. Clicando duas vezes na caixa de displays permite ver o circuito gerado para esse módulo.



No relatório de compilação é possível verificar a quantidade de recursos usados do dispositivo:

Flow Summary	
Flow Status	Successful - Fri Mar 11 15:48:16 2022
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	test1
Top-level Entity Name	toplevel
Family	MAX 10
Device	10M50DAF484C6GES
Timing Models	Preliminary
Total logic elements	42 / 49,760 (< 1 %)
Total registers	0
Total pins	65 / 360 (18 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

Para a gravação, acesse o menu *Tools => Programmer*. A tela a seguir deve ser mostrada (lembre de conectar a placa via USB)³. Verifique se a USB-Blaster está selecionada (em destaque na figura).



Caso a USB-Blaster não esteja selecionada, clique no botão *Hardware Setup* à esquerda e selecione a [USB-0]. Agora basta clicar em *Start* e a gravação deve ser rápida pois ela ocorre na memória volátil para o arquivo com extensão *.sof* (verifique se esse arquivo é o da pasta para o qual o projeto foi descompactado). A barra de progresso deve ficar verde (*Successful*).

Imediatamente os leds devem mudar o padrão de piscagem – nosso projeto de exemplo é um pouco diferente daquele que está gravado na memória não-volátil da placa. Teste as chaves pra ver se está tudo certo. Se a placa for desenergizada nossa nova programação se perde.

Tarefas

Modifique o projeto para implementar as tarefas a seguir e **APRESENTE AO PROFESSOR CADA UM DOS ITENS ASSIM QUE ELE FUNCIONAR**.

1. No próprio projeto exemplo, alterar e testar a lógica entra as chaves e push buttons para os LEDs e displays da placa;
2. Implementar uma ULA combinacional de 4 bits de largura e 3 operações utilizando como as duas entradas 8 chaves de duas posições e como saídas 4 Leds para o resultado e outros LEDs para os flags. Ligue também os displays de 7 segmentos para o resultado da conta.
3. (extra) Medir com um osciloscópio o tempo de propagação do LSb pra o MSb e para os flags da ULA (talvez seja necessário alargar os barramentos de 4 para 32 bits). Os pinos de medição podem ser mapeados para o conector de GPIO 2x20 da DE10-Lite.

Obs: Pode alterar diretamente o *toplevel.vhd* e, se desejar, pode incluir novos arquivos no projeto, da mesma forma que vem sendo feito nos labs. Se necessário, consulte o manual da placa para remapear os sinais para outros pinos

3 Caso isso não funcione, é provável que o *driver* do programador não tenha sido instalado.